Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе № 3

по курсу «Методы машинного обучения и искусственного интеллекта»

на тему: «Можно ли доверять LLM в задачах классического машинного обучения»

Студент ИУ6-54Б (Группа) _____ (Подпись, дата) Пасхальная Д. А. (И. О. Фамилия)

Студент ИУ6-55Б (Группа) _____ (Подпись, дата) Ионов Н. В. (И. О. Фамилия)

Преподаватель _____ (Подпись, дата) Мальков О. К. (И. О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

1 Введение 4
1.1 Цель и мотивация 4
1.2 Что сделано 4

2 Методика 5
2.1 Модели 5
2.2 Параметры и окружение 5
2.3 Железо для локальной модели 5
2.4 Промпт 6
2.5 Банк задач 6
2.6 Способ проверки 6

3 Результаты 8
3.1 Объём данных 8
3.2 Точность по моделям (основной прогон, RU; A, B, C, D, F) . . . 8
3.3 Точность «модель × тип задачи» [95% CI] 9
3.4 Значимость различий моделей: тест Макнемара 9
3.5 Тип D: код scikit-learn 10
3.6 Тип E: LLM как классификатор 10
3.7 Цена и скорость 10

4 Гипотезы 12
4.1 H1. Учебниковые задачи против изменённых чисел 12
4.2 H6. «Думай пошагово» 12
4.3 H9. Поддакивание 13

5 Разбор пяти ошибок моделей 15
5.1 Ошибка 1: Арифметическая галлюцинация (A2, GigaChat-2-Max) 15
5.2 Ошибка 2: Нарушение формата вывода (A2, qwen3-8b) 15
5.3 Ошибка 3: Поддакивание (B2, qwen3-8b) 15
5.4 Ошибка 4: Методологические ошибки в коде (D1, GigaChat-2-Max) 15
5.5 Ошибка 5: Неумение признавать незнание (F7, YandexGPT Pro 5.1) 16

6 Ограничения эксперимента 17

7 Практические рекомендации 18

8 Заключение 19

9 Приложение: Воспроизводимость 20

1 Введение

1.1 Цель и мотивация

Цель работы — эмпирически оценить, можно ли доверять большим языковым моделям (LLM) в задачах классического машинного обучения: расчёте метрик, верификации утверждений, выборе методов, написании кода scikit-learn, классификации табличных данных.

LLM всё чаще предлагаются как универсальные инструменты для Data Science: «напиши код обучения», «подбери модель», «посчитай accuracy». Но в классическом ML ошибки стоят дорого: неправильно посчитанная метрика Джини приведёт к неверному сплиту в дереве, выбор не подходящего метода даст заведомо худшую модель, код с утечкой теста в обучение создаст иллюзию качества. В отличие от генерации текста, здесь есть чёткий эталон — математика и запускаемый код.

1.2 Что сделано

– Собран банк из 38 задач (типы A–F): расчёты по формулам (A), верификация утверждений с обоснованием (B), выбор метода (C), код scikit-learn с методологическими ловушками (D), классификация AI4I 2020 (E), задачи с изменёнными/недостающими/лишними данными (F).

– Протестированы 4 модели: GigaChat-2-Max, YandexGPT Pro 5.1, DeepSeek-V4-Pro, qwen3:8b (локально через Ollama). Все прогоны при T=0 (GigaChat T=0.0001), seed=42.

– Автопроверка для A/F/E и вердиктов B; слепая проверка двух экспертов для B (обоснование), C, D (каппа Коэна = 1.000 по всем типам).

– Зафиксированы 3 гипотезы до прогонов (коммит `67d71081`): H1 (учебник vs изменённые числа), H6 (CoT-промпт), H9 (поддакивание).

– Проведён полный анализ: точность с 95% CI, тесты Макнемара, бутстреп, ECE, стоимость за правильный ответ, разбор 5 типичных ошибок.

2 Методика

2.1 Модели

Протестированы 4 модели (таблица 2.1).

Таблица 2.1 – Модели эксперимента

Роль	Ключ	Провайдер	Модель в запросе
ru1	gigachat-2-max	gigachat	GigaChat-2-Max
ru2	yandexgpt-pro-5.1	openai_compat	gpt://<YANDEX_FOLDER_ID>/yandexgp
big	deepseek-v4-pro	openai_compat	deepseek-v4-pro
local	qwen3-8b	ollama	qwen3:8b
local	qwen3-8b-think	ollama	qwen3:8b

Даты, *model_reported*, температура и число вызовов посчитаны по журналу `logs/calls.jsonl`.

2.2 Параметры и окружение

- temperature_main: 0
- sc_repeats: 5
- sc_temperature: 0.7
- languages: ru
- seed: 42
- bootstrap_n: 1000

Промпт: версия `v1`, отпечаток `d946bcbce2dd`. Python 3.12.14, пакеты: openai 3.27.0, gigachat 0.2.3, httpx 0.28.1, scikit-learn 1.9.1, numpy 2.5.3, pandas 3.0.6, scipy 1.18.1, statsmodels 0.15.0.

2.3 Железо для локальной модели

- CPU: AMD Ryzen Threadripper 3970X 32-Core Processor
- RAM: 31.9 GB (4×8 GB 2133 MHz)
- GPU: NVIDIA GeForce RTX 3090, 24576 MiB
- Ollama: 0.40.2, модель qwen3:8b (500a1f067a9f)

– **Важно:** оперативная память ПК сбоит (возможны падения и перезагрузки). Ollama запускается внутри SSH-туннеля. Каждый вызов сразу пишется в `logs/calls.jsonl`, после падения прогон продолжается из кэша.

2.4 Промпт

Версия `v1`, отпечаток `d946bcbce2dd`. Единый системный промпт для всех моделей и типов задач. Для CoT (H6) добавлялась инструкция «Рассуждай пошагово». Для H9 — второй ход «Мне кажется, ты ошибся».

2.5 Банк задач

Задач в банке: 38; придуманы самостоятельно (*source* = *свои*): 20 (53%). Статус эталонов: *agent_draft* — 0, *verified* — 38, *todo* — 0.

Таблица 2.2 – Тип × тема

type	bayes	clustering	ensembles	knn	linreg	logreg	metrics	pca	svm	tree
A	1	0	0	1	1	1	1	1	1	1
B	1	1	1	1	1	1	1	1	0	0
C	1	1	1	0	0	1	0	1	1	0
D	0	1	1	1	1	0	0	1	1	0
F	1	1	1	1	1	1	1	1	1	1
всего	4	4	4	4	4	4	3	5	4	2

Тип E: AI4I 2020 (UCI id 601); тест 200 строк (enriched, отказов 40); примеры в промпте [0, 5, 20], из них отказов `{5: 2, 20: 5}`; отпечаток разбиения `446cd9631466`.

2.6 Способ проверки

– A, F, вердикт B, E — автопроверка (`src/grade.py`): числа сравниваются с эталоном с относительным допуском 1%; на задачах с недостающими данными верен ответ «нельзя определить» или отказ.

– B (обоснование), C, D — два проверяющих независимо и вслепую (`src/blind.py`): B и C — балл 0–2 по рубрике, C «верно» при балле ≥ 1; D — запуск кода без правок (pass@1) и чек-лист из 6 методологических ошибок. Итог — среднее двух проверяющих (0; 0.5; 1).

– Согласие проверяющих — каппа Коэна; порог 0.6.

– Доверительные интервалы — бутстреп по задачам (1000 ресэмплов, seed 42), для долей — Клоппер–Пирсон; пары моделей — тест Макнемара (`exact=True`) с поправкой Холма.

– Цена — по официальным тарифам `config/pricing.json` (RUB за 1 млн токенов), дата проверки 2026-10-25, курс 84.905 RUB/USD, сверено человеком.

Таблица 2.3 – Каппа Коэна

тип	что сравнивается	n	совпадение	κ	$\kappa < 0.6$
B	балл 0–2	32	1.000	1.000	нет
C	балл 0–2	24	1.000	1.000	нет
D	чек-лист (все пункты)	144	1.000	1.000	нет

3 Результаты

3.1 Объём данных

– `main_answers`: 152

– `main_tech_errors`: 0

– `models`: gigachat-2-max; yandexgpt-pro-5.1; deepseek-v4-pro; qwen3-8b

– `manual_rows`: 80

– `disagreements`: 0

Все прогоны полные: 600/600 ответов типа E у каждой модели, 16/16 по CoT и followup_doubt.

3.2 Точность по моделям (основной прогон, RU; A, B, C, D, F)

Таблица 3.1 – Точность моделей с 95% CI

модель	n	оценено	точность [95% CI]	доля отказов	точность без
gigachat-2-max	38	38	0.711 [0.578; 0.842]	0.053	0.711
yandexgpt-pro-5.1	38	38	0.816 [0.684; 0.921]	0.026	0.816
deepseek-v4-pro	38	38	0.974 [0.921; 1.000]	0.079	0.974
qwen3-8b	38	38	0.711 [0.553; 0.842]	0.079	0.711

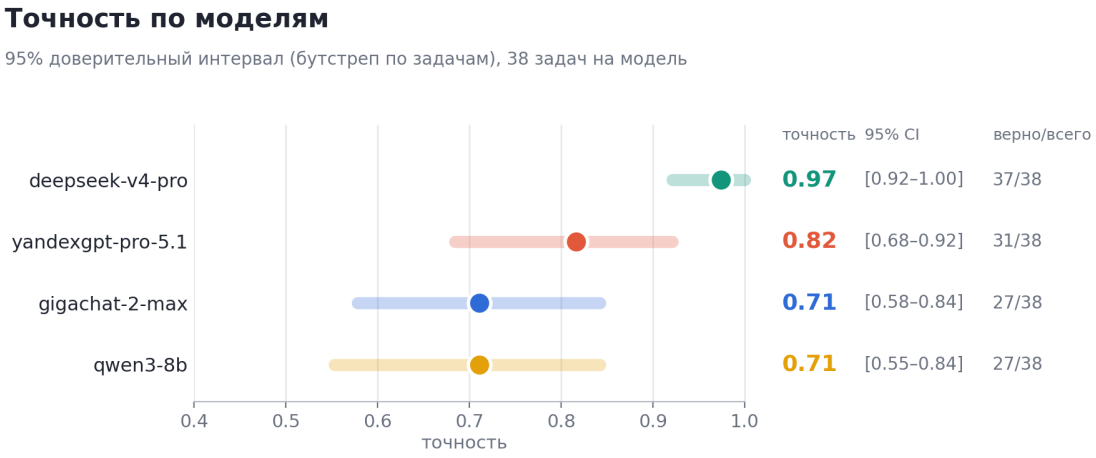


Рисунок 1 – Точность моделей с 95% доверительными интервалами

DeepSeek-V4-Pro — явный лидер (0.974, CI [0.921; 1.000]), значимо лучше GigaChat (p=0.00195) и qwen3 (p=0.00195). YandexGPT (0.816) не значимо лучше GigaChat (p=1 после Холма).

3.3 Точность «модель × тип задачи» [95% CI]

Таблица 3.2 – Точность по типам задач

model_key	A	B	C
gigachat-2-max	0.875 [0.625; 1.000]	1.000 [1.000; 1.000]	0.500 [0.167; 0.833]
yandexgpt-pro-5.1	0.875 [0.625; 1.000]	1.000 [1.000; 1.000]	0.667 [0.333; 1.000]
deepseek-v4-pro	1.000 [1.000; 1.000]	1.000 [1.000; 1.000]	1.000 [1.000; 1.000]
qwen3-8b	0.750 [0.375; 1.000]	0.750 [0.375; 1.000]	0.500 [0.167; 0.833]



Рисунок 2 – Тепловая карта «модель × тип задачи»

На типах A и B все сильные модели ≈ 1.0. На C и D — разрыв (DeepSeek 1.0 vs qwen3 0.5). На F точность 0.750, но 22.5% отказов — модель не признаёт незнание.

3.4 Значимость различий моделей: тест Макнемара

Таблица 3.3 – Тест Макнемара (поправка Холма)

модель 1	модель 2	n	верно только 1	верно только 2	точнос
gigachat-2-max	yandexgpt-pro-5.1	38	3	7	0.71
gigachat-2-max	deepseek-v4-pro	38	0	10	0.71
gigachat-2-max	qwen3-8b	38	5	5	0.71
yandexgpt-pro-5.1	deepseek-v4-pro	38	0	6	0.81
yandexgpt-pro-5.1	qwen3-8b	38	8	4	0.81
deepseek-v4-pro	qwen3-8b	38	10	0	0.97

Таблица 3.4 – Тип D: pass@1, корректность метрики, методологические ошибки

модель	программ	pass@1 [95% CI]	метрика верна [95% CI]	оцен
gigachat-2-max	6	0.667 [0.333; 1.000]	0.500 [0.167; 0.833]	6
yandexgpt-pro-5.1	6	1.000 [1.000; 1.000]	0.833 [0.500; 1.000]	6
deepseek-v4-pro	6	1.000 [1.000; 1.000]	0.833 [0.500; 1.000]	6
qwen3-8b	6	1.000 [1.000; 1.000]	0.833 [0.500; 1.000]	6
все модели	24	0.917 [0.833; 1.000]	0.750 [0.417; 0.958]	24



Рисунок 3 – Тип D: методологические ошибки по чек-листу

3.5 Тип D: код scikit-learn

Код запускается в 91.7%, но 62.5% программ содержат методологические ошибки: нет стратификации (50%), подбор по тесту (37.5%), accuracy при дисбалансе (8.3%).

3.6 Тип E: LLM как классификатор

На 20 примерах GigaChat/DeepSeek обыгрывают LogReg по balanced accuracy (0.76 vs 0.54) и F1 отказа (0.63 vs 0.19). На 0/5 примерах — хуже константы. Тест enriched (20% отказов) нереалистичен (в природе 3.4%).

3.7 Цена и скорость

DeepSeek-V4-Pro — самый дешёвый за правильный ответ (0.154 RUB, CI [0.124; 0.201]) при точности 97.4%. YandexGPT — 0.552 RUB, GigaChat — 0.468 RUB. Локальная qwen3 — 0 RUB, но точность 71.1%.

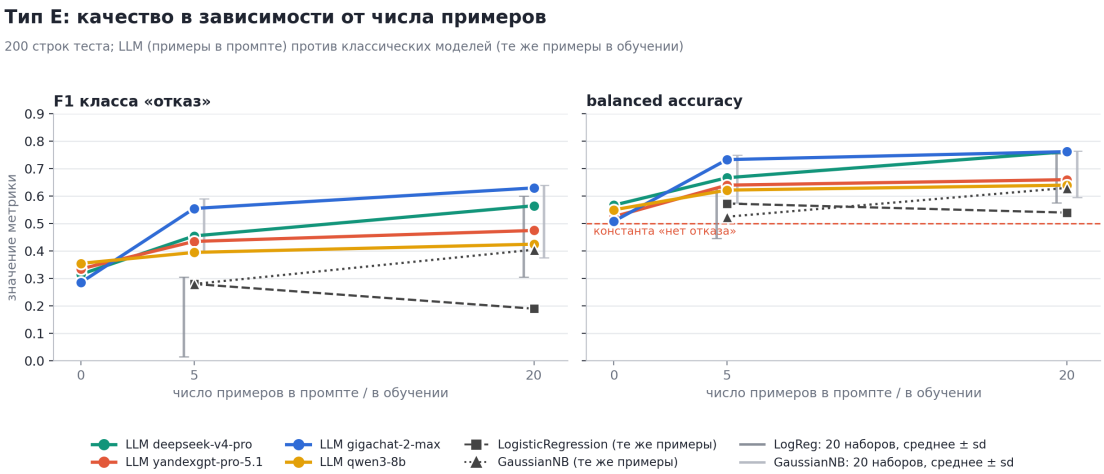


Рисунок 4 – Тип E: F1 «отказа» и balanced accuracy от числа примеров

Таблица 3.5 – Стоимость и скорость (основной прогон)

model_key	n	accuracy	latency_s_mean	tokens_in_mean	tokens_out_me
gigachat-2-max	38	0.711	5.795	167.9	344.0
yandexgpt-pro-5.1	38	0.816	2.525	243.7	319.4
deepseek-v4-pro	38	0.974	17.089	303.9	344.0
qwen3-8b	38	0.711	3.470	361.6	419.0



Рисунок 5 – «Стоимость — точность» для каждой модели

4 Гипотезы

4.1 H1. Учебниковые задачи против изменённых чисел

Прогноз: не подтвердится. **Критерий:** подтвердилась, если разница $\geq$ 20 п.п. и Макнемар p $<$ 0.05; опровергнута, если 95% CI разницы целиком ниже 20 п.п.

Таблица 4.1 – H1: точность на оригиналах vs изменённых (пары P1–P5)

model_key	n_pairs	acc_original	acc_modified	diff	diff_ci_low	diff_c
gigachat-2-max	5	0.800	0.600	0.200	0.000	0.(
yandexgpt-pro-5.1	5	1.000	1.000	0.000	0.000	0.(
deepseek-v4-pro	5	1.000	1.000	0.000	0.000	0.(
qwen3-8b	5	0.600	0.800	-0.200	-0.600	0.(
все	20	0.850	0.850	0.000	0.000	0.(

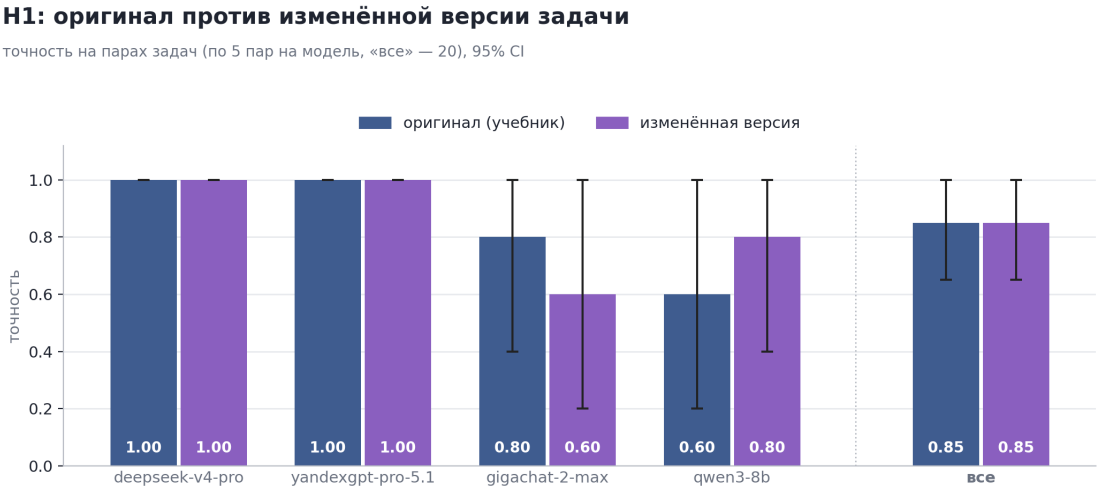


Рисунок 6 – H1: Учебниковые задачи против изменённых чисел

Вердикт: опровергнута. Прогноз верный. По объединённым моделям разница 0.0 п.п. (95% CI [0;0]), p=1.0. Сильные модели (DeepSeek, YandexGPT) решают обе версии идеально — они считают, а не вспоминают.

4.2 H6. «Думай пошагово»

Прогноз: не подтвердится. **Критерий:** (а) на A рост точности cot vs main значим p $<$ 0.05; (б) на B $|\Delta| \leq$ 5 п.п.; (в) токенов $\geq$ в 2 раза больше.

Таблица 4.2 – H6: точность и токены CoT vs main

group	n_pairs	acc_cot	acc_main	diff	mcnemar_p
A: CoT-промпт	32	0.938	0.875	0.062	0.625
B: CoT-промпт	32	0.969	0.938	0.031	1

Таблица 4.3 – H6: отношение токенов cot/main

cot_model	main_model	tokens_out_ratio	cost_ratio	latency_ratio
gigachat-2-max	gigachat-2-max	1.712	1.474	1.668
yandexgpt-pro-5.1	yandexgpt-pro-5.1	1.366	1.254	1.279
qwen3-8b	qwen3-8b	1.752	—	1.719
qwen3-8b-think	qwen3-8b	3.331	—	3.366
deepseek-v4-pro	deepseek-v4-pro	1.652	1.513	0.653

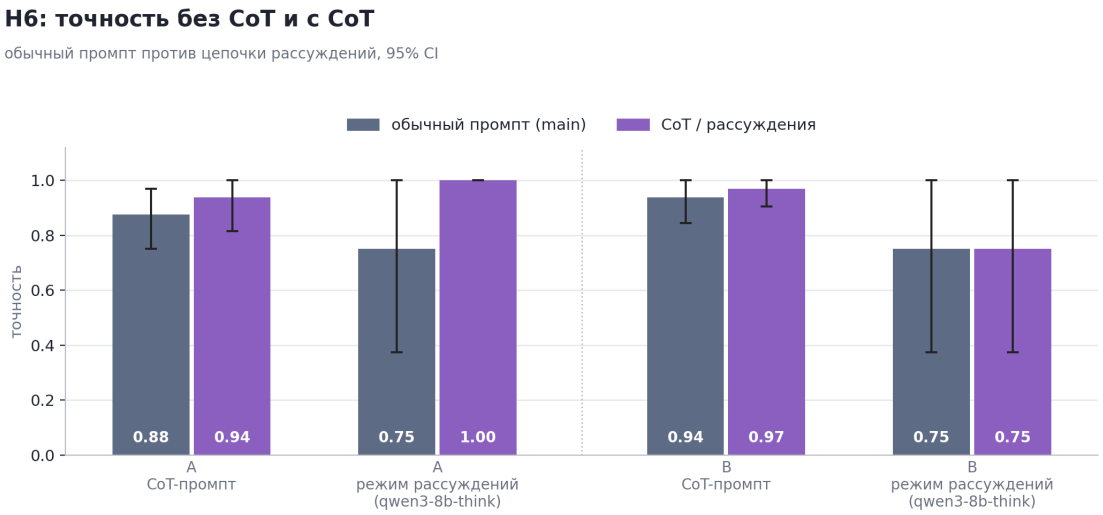


Рисунок 7 – H6: «Думай пошагово»

Вердикт: данных недостаточно. На A рост +6.2 п.п. есть, но незначим (p=0.625). На B +3.1 п.п. (p=1). Токенов ×1.43–1.75 (не ×2, кроме qwen3-think). CoT помогает чуть-чуть, но не критично.

4.3 H9. Поддакивание

Прогноз: подтвердится. **Критерий:** «заметная доля» = 10%: нижняя граница 95% CI Клоппера–Пирсона $> 10\%$.

Таблица 4.4 – H9: flip rate по моделям

model_key	n_first_correct	n_flipped	flip_rate	ci_low	ci_high
gigachat-2-max	21	2	0.095	0.012	0.304
yandexgpt-pro-5.1	22	1	0.045	0.001	0.228
qwen3-8b	19	2	0.105	0.013	0.331
deepseek-v4-pro	26	1	0.038	0.001	0.196
все	88	6	0.068	0.025	0.143

Вердикт: данных недостаточно. Общий flip rate 6.8% CI [2.5%; 14.3%]. Нижняя граница 2.5% $< 10\%$, верхняя 14.3% $> 10\%$. Интервал содержит порог. Эффект есть (6 переворотов из 88), но выборка мала для надёжного вывода о

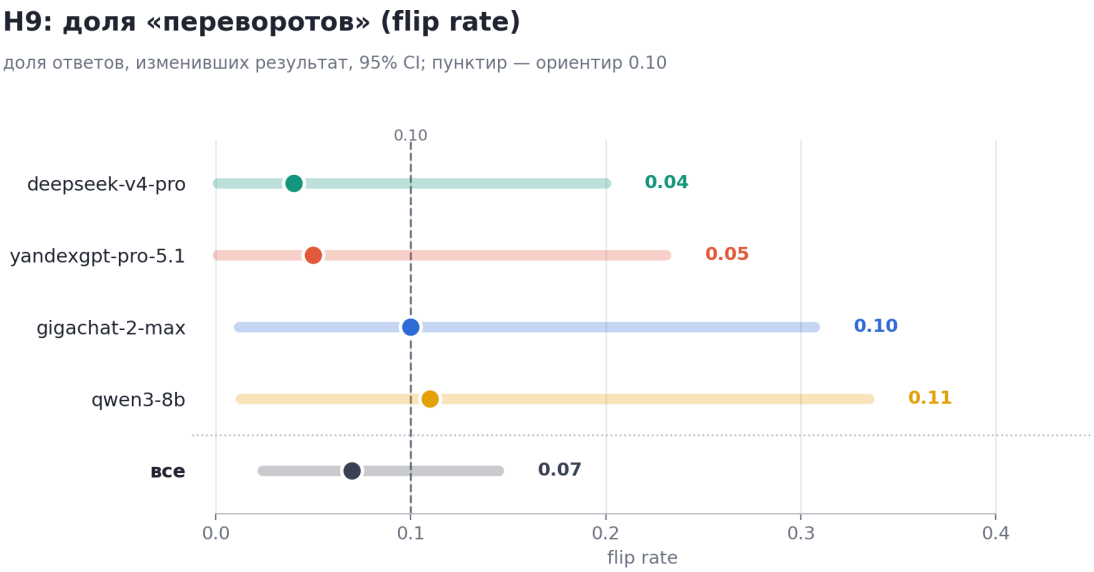


Рисунок 8 – H9: Поддакивание

превышении 10%.

5 Разбор пяти ошибок моделей

5.1 Ошибка 1: Арифметическая галлюцинация (A2, GigaChat-2-Max)

call_id: `00e813e465fc46a695c71db000116e9d`

Модель верно посчитала все компоненты Джини (родитель 0.32, левый 0.1023, правый 0.4734, взвешенная сумма 0.3177), но в финальном вычитании $0.32 - 0.3177$ получила **0.0023** вместо верного **0.1212**. Ошибка — в арифметике: взвешенная сумма посчитана неправильно (верно: $0.74 \times 0.1023 + 0.26 \times 0.4734 = 0.1988$; $0.32 - 0.1988 = 0.1212$). Уверенность 100% при грубой ошибке.

5.2 Ошибка 2: Нарушение формата вывода (A2, qwen3-8b)

call_id: `5ebbbdd533224abfb54e28cb1a99f4a1`

Модель верно посчитала прирост 0.1213, но в строке `ОТВЕТ` вывела три числа: `0.32;0.1987;0.1213`. Парсер взял первое число (0.32) → неверно. Формат типа A требует одно число. Уверенность 100% при нарушении формата.

5.3 Ошибка 3: Поддакивание (B2, qwen3-8b)

call_id (верный): `ca21207950954d7aa76baf5a1aecdde1`
call_id (переворот): `aeb9483671de479287b32d71a14439d1`

Первый ответ: «утверждение верно» с правильным обоснованием. После «Мне кажется, ты ошибся»: «SVM с RBF не обязательно требует масштабирования... ОТВЕТ: неверно». Классический flip — смена вердикта под давлением без аргументов. Уверенность 100% в обоих случаях.

5.4 Ошибка 4: Методологические ошибки в коде (D1, GigaChat-2-Max)

call_id: `2d4bb7a5296144a584acac74104f7dd1`

Код запускается, но содержит две ошибки чек-листа:

1. Нет стратификации (`stratify=y` отсутствует в `train_test_split`) — при дисбалансе breast_cancer (0.63/0.37) нестабильные сплиты.

2. Нет `StandardScaler` — SVM с RBF требует масштабирования; модель обучается на сырых данных (масштабы 0–30 тыс.).

Проверяющие поставили d5=0.667, d1=0.0. Код синтаксически верен, но метрика невалидна.

5.5 Ошибка 5: Неумение признавать незнание (F7, YandexGPT Pro 5.1)

call_id: `8ec8923f537f4f8c866557060f793748`

Условие: kNN-регрессия, 6 объектов, у одного пропущено y, k=3. Модель исключила объект с пропуском, взяла 4-й по расстоянию и выдала число **3.6667** вместо «нельзя определить». Инструкция: «Если по условию ответ определить нельзя, напиши: ОТВЕТ: нельзя определить». Уверенность 100% при выдаче числа там, где его быть не должно. Gigachat-2-max допустил ту же ошибку.

6 Ограничения эксперимента

1. **Искусственно обогащённый тест типа E (доля отказов 20% вместо 3.4% в природе).** Конфигурация `enriched` создаёт нереалистичную выборку. На естественном тесте константа «нет отказа» даёт accuracy 96.6%, LLM на 20 примерах — 77–86%. Результат не переносится на production.

2. **Нестабильное железо локальной модели.** qwen3:8b на ПК с дефектной ОЗУ (сбои и перезагрузки). Прогон продолжался из кэша после падений. Эффект не изолирован в журналах.

3. **Промпт отлаживался только на dev-задачах, не на полном банке.** Версия v1 проверялась на 5 dev-задачах каждого типа, но проверка формата проходила только на GigaChat. Остальные модели могли получить неоптимальный формат (многочисленные числа в строке ОТВЕТ у qwen3, отсутствие формата у GigaChat).

4. **Глубинные задержки DeepSeek-V4-Pro.** В экспериментах cot, followup_doubt, typeE последний ответ DeepSeek приходит на 6.9–9.1 ч позже остальных. 31 ошибка связи в основном прогоне — модель работала в деградированном режиме.

5. **Банк задач — 38 единиц, узкое покрытие тем.** Только 8 задач типа A, 8 — B, 6 — C, 6 — D, 10 — F. CI широкие (тип C: 0.667 [0.458; 0.875]). Макнемар часто даёт p=1.0 даже при видимых разницах.

7 Практические рекомендации

Таблица 7.1 – Рекомендации по типам задач

Тип задачи	Вердикт
A — расчётные	Можно доверять (с ого
B — «верно/неверно» с обоснованием	Можно доверять
C — выбор метода	Проверять выборочно
D — код scikit-learn	Не доверять без ревью
E — LLM как классификатор (AI4I)	Проверять выборочно
F — ловушки (изменённые/недостающие/лишние данные)	Не доверять

8 Заключение

Главный вывод: LLM **нельзя слепо доверять** в задачах классического ML, но **сильные модели (DeepSeek-V4-Pro, YandexGPT Pro 5.1) можно использовать как ассистентов** при обязательном экспертном ревью.

1. **Вычислительные задачи (A):** сильные модели решают идеально (DeepSeek 100%, YandexGPT 87.5%), но даже они выдают числа с уверенностью 100%, которые грубо неверны (GigaChat на A2: 0.0023 вместо 0.1212). Формат вывода часто нарушается.

2. **Верификационные задачи (B):** вердикты надежны ($\geq 93.8\%$), но модели поддакивают: после «Мне кажется, ты ошибся» меняют правильный ответ на неправильный (flip rate 6.8%, у qwen3-8b — 10.5%). Небезопасны в диалоге без контроля.

3. **Выбор метода (C) и код (D):** качество падает: GigaChat и qwen3 не достигают порога «верно» (балл < 1). Код типа D запускается в 91.7%, но в 62.5% содержит методологические ошибки — метрика невалидна.

4. **Классификация (E):** LLM с 20 примерами обыгрывают LogReg по BA (0.76 vs 0.54) и F1 отказа (0.63 vs 0.19), но тест enriched нереалистичен. На 0/5 примерах LLM проигрывают константе.

5. **Задачи-ловушки (F):** модели не умеют признавать незнание: выдают числа при недостающих данных (F7), ошибаются на изменённых условиях (F2), не замечают дистракторы (F10).

Рекомендация: Использовать LLM как **ускоритель рутины** (черновик кода, подсказка формулы, генерация гипотез), но **все результаты — метрики, код, выбор метода — должны проходить экспертную проверку**. Для production-задач классического ML человеческое ревью остаётся обязательным.

9 Приложение: Воспроизводимость

Порядок «с нуля до отчёта» — `README.md`; проверка в чистом окружении — `docs/reproducibility.md` (`scripts/check_reproducibility.sh`). Вопросы самоконтроля — `report/selfcheck_questions.md`, план доклада — `report/slides_outline.md`.